All-Sky Autonomous Computing in UAV Swarm

Hao Sun, Yuben Qu, Chao Dong, Haipeng Dai, Zhenhua Li, Lei Zhang, Qiuhui Wu, Song Guo

IEEE Transactions on Mobile Computing, Vol. 23, No. 12, December 2024

论文汇报

汇报人：李钱　　导师：吴鹏飞

汇报时间：2026.6.26

论文基本信息与汇报框架

All-Sky Autonomous Computing in UAV Swarm

Hao Sun, Yuben Qu, *Member, IEEE*, Chao Dong, *Senior Member, IEEE*,
Haipeng Dai, *Senior Member, IEEE*, Zhenhua Li, *Senior Member, IEEE*, Lei Zhang,
Qihui Wu, *Fellow, IEEE*, and Song Guo, *Fellow, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs) play an essential role in emergency cases and adverse environments for applications like disaster detection and mine exploration. To process the massive volume of sensing data generated by various sensory payloads in these applications, existing works either compress deep learning (DL) models to conduct onboard computing, or offload raw data back to the resourceful ground station with the help of relay UAVs due to base station damage. However, the former sacrifices the inference accuracy of DL models (up to 10% accuracy loss), while the latter achieves high accuracy at the cost of significant latency, due to limited wireless communication resources in the multi-hop transmission. To address the problem, exploiting the resources of the UAV swarm including both task UAVs and relay UAVs, we build up an all-sky autonomous computing (ASAP) system to autonomously conduct collaborative computing in the swarm, to achieve both high accuracy and low latency of sensing data processing. In detail, we first propose a novel UAV swarm-native collaborative computing architecture, considering the general hierarchy and clustering structure of UAV swarms, as well as the characteristic of DL model execution. We then design an elastic efficient task scheduler to allocate computing tasks for UAVs, and update the scheduling scheme online when some UAVs are unavailable, with the aid of a lightweight and accurate DL inference performance predictor. Finally, we design an adaptive inter-UAV data compressor, to adapt to the limited and dynamic communication resources between UAVs. Experiment results on 24 airborne computers and five real-world UAVs show that, the proposed system can perform collaborative computing in a timely manner and effectively deal with situations when some UAVs become unavailable.**

***Index Terms*—UAV swarm, collaborative inference, model inference performance prediction, task scheduling.**

I. INTRODUCTION

DRONES, or unmanned aerial vehicles (UAVs) have a wide range of critical applications in emergency and harsh environments that are difficult for humans to navigate. Some examples include earthquake search and rescue [1], [2], forest fire detection [3], [4], and mine exploration [5], [6], [7]. To support the desired capabilities of the aforementioned applications, some sensory payloads need to be deployed on UAVs, e.g., a variety of EO/IR cameras and radars, which usually generate abundant sensing data [8], while the communication in the aforementioned environment is generally restricted or even damaged. Therefore, how to process these data effectively and efficiently is critical to the applications.

Due to the outstanding performance of deep learning (DL), most of these applications employ DL techniques for fast computational processing [9], [10]. Nevertheless, these DL models are usually computationally expensive, e.g., processing a frame of 720P video with ResNet101 model requires a computational cost of 14.47 GFLOPs (FP32) and a memory of 3.99 GB in our field test. Because of the load and power limitations, both airborne computation and memory resources of UAVs are limited, e.g., the frequently used Nvidia Jetson Nano airborne computer only has approximately 7.6 GPLOFS of computing power under FP32 and 4 GB of memory [11], and the aforementioned computing task cannot be completed because not all the memory can be used for computing. Therefore, conducting computational processing of the sensory payload data on a single UAV with accurate but complex DL models is tough.

To relieve the computational burden, there are generally two main approaches as follows. 1) *Model compression:* Many works try to reduce computing overhead by the lightweight design of model structure (line ❸ in Fig. 1) via model pruning [12], [13], model quantization [14], [15], knowledge distillation [16], [17] and neural architecture search [18], [19], which could decrease the inference accuracy by up to 10% [20], [21], [22]. 2) *Raw data offloading:* Most existing UAV systems transmit the captured on-site raw data back to the resourceful ground station for highly accurate data processing [8], [23], [24], [25], [26]. Due to the wide UAV working area, the data cannot be directly transmitted back to the remote ground station, which usually requires some base stations for communication relay (line ❹ in Fig. 1). However, the communication relay is usually unavailable in these tough environments, due to base station damage or lack of conditions for installing communication equipment. To this end, some works conduct communication relay on UAVs instead of base stations [27], [28], [29], [30], [31] (line ❷ in Fig. 1). Nevertheless, the wireless communication resources in the UAV swarm are still limited, which results in high transmission latency when transmitting raw images. Overall, existing

Manuscript received 9 February 2024; revised 14 May 2024; accepted 7 July 2024. Date of publication 15 July 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61931011 and Grant 62072303. Recommended for acceptance by X. Peng. *(Corresponding author: Chao Dong.)*

Hao Sun, Yuben Qu, Chao Dong, Lei Zhang, and Qihui Wu are with the Key Laboratory of Dynamic Cognitive System of Electromagnetic Spectrum Space, Ministry of Industry and Information Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210023, China (e-mail: sunhaosn@nuaa.edu.cn; quyuben@nuaa.edu.cn; dch@nuaa.edu.cn; Zhang_lei@nuaa.edu.cn; wuqihui@nuaa.edu.cn).

Haipeng Dai is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: haipengdai@nju.edu.cn).

Zhenhua Li is with the School of Information Science and Technology, Tsinghua University, Beijing 100190, China (e-mail: lizhenhua1983@tsinghua.edu.cn).

Song Guo is with the Hong Kong University of Science and Technology, Hong Kong 999077, China (e-mail: songguo@cse.ust.hk).

Digital Object Identifier 10.1109/TMC.2024.3427420

论文信息：

- ➢ 期刊：IEEE Transactions on Mobile Computing (TMC)
- ➢ 时间：2024 年 12 月；主题：UAV swarm 中的自主协同计算
- ➢ **核心系统：ASAP (All-Sky Autonomous Computing)，在空中完成高精度、低时延推理**

汇报结构：

- 1. 背景问题与研究动机
- 2. ASAP 系统架构与三类核心模块
- 3. 调度器、预测器、压缩器的技术细节
- 4. 系统实现与实验结果
- 5. 总结、局限与启发

论文的背景和问题：

- 无人机常用于地震搜救、森林火灾、矿井探索等复杂环境，能够采集大量现场图像与传感数据。
- 资源受限：机载设备算力、显存和功耗有限，复杂 DL 模型很难单机实时推理。
- 地面依赖不可靠：灾区基站可能损坏，原始数据回传会产生多跳传输和长时延。
- **现有折中方案要么压缩模型牺牲精度，要么转发原始数据牺牲时延。**

核心矛盾：高精度推理 vs. 低时延响应

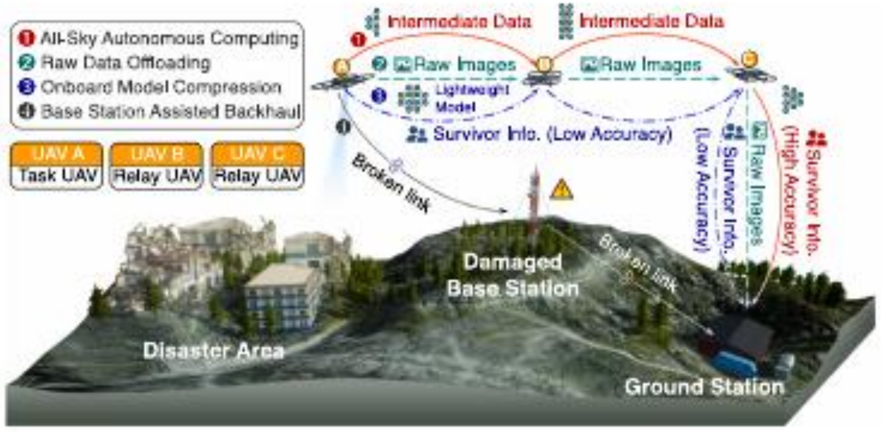


Fig. 1. A killer application of UAVs: earthquake search and rescue. Due to the possible broken base station communication link (line ❹), existing relay UAV assisted approaches either transmit the massive on-site images back to the ground station for further processing in a *time-consuming* manner (line ❷), or analyse the images onboard with a compressed DL model in *low accuracy* (line ❸)

图1：地震搜救场景中，ASAP 选择在 UAV swarm 内部协同计算，只把高价值结果返回地面。

论文的动机和贡献：

三个挑战

- 如何保证 UAV swarm 协同计算的效率与稳定性：设备异构、节点掉线、集群故障都会影响分配。
- 如何快速评估任务代价：在线调度需要轻量、准确的 DL 推理性能预测。
- 如何应对有限且动态的 UAV 间通信资源：通信速率随移动与链路质量变化。

主要贡献

- 提出 UAV swarm 原生协同计算架构：跨集群切模型，集群内切数据。
- 设计 Elastic Efficient Scheduler：两层负载均衡 + 弹性重调度，适配节点不可用。
- 设计轻量推理时延预测器与自适应中间数据压缩器，在准确率和效率之间取得平衡。

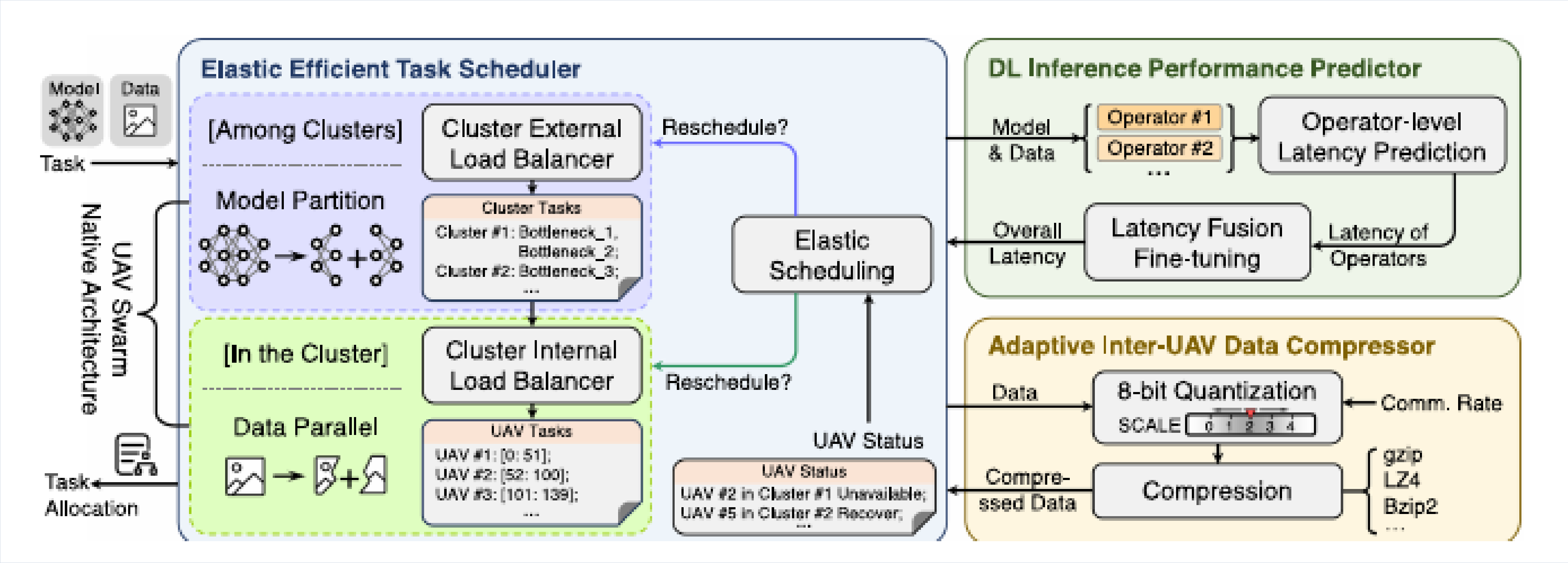
目标：让 UAV swarm 在空中自主、快速、可靠地完成传感数据推理

相关工作：现有范式为什么不够?

范式	主要思路	在 UAV swarm 中的问题	ASAP 的处理
模型压缩	剪枝 / 量化 / 蒸馏	降低计算量，但可能带来最高约 10% 精度损失	**不压缩模型结构，保持高精度**
原始数据卸载	把图像回传地面计算	链路可能中断，且多跳传输时延高	**在空中处理，只回传结果**
地面边缘协同	模型切分或数据切分	通常假设稳定网络或中心节点，不适合集群组织	**跨集群模型分区 + 集群内数据并行**

论文定位：ASAP 并不是单纯做任务卸载，而是针对 UAV swarm 的组织结构、通信波动和设备异构，重新设计协同推理流程。

ASAP 系统总览：三模块协同



- Elastic Efficient Task Scheduler：负责模型分区、数据分区和失效后的重调度。
- DL Inference Performance Predictor：为在线调度提供每个模型块/数据块的时延估计。
- **Adaptive Inter-UAV Data Compressor：根据数据规模和通信速率动态压缩中间特征。**

UAV Swarm 原生协同计算架构

核心思想：模型和数据分两层切分

- 跨集群：将完整 DL 模型切成多个 submodel，由不同 UAV cluster 形成流水线。
- 集群内：每个 cluster head 将输入数据切成若干块，分发给成员 UAV 并行计算。
- 中间结果：上一集群输出作为下一集群输入，在 cluster head 之间传输。

- **优势：避免把所有原始图像发回地面，也避免在同一集群内切模型导致过多 relay。**

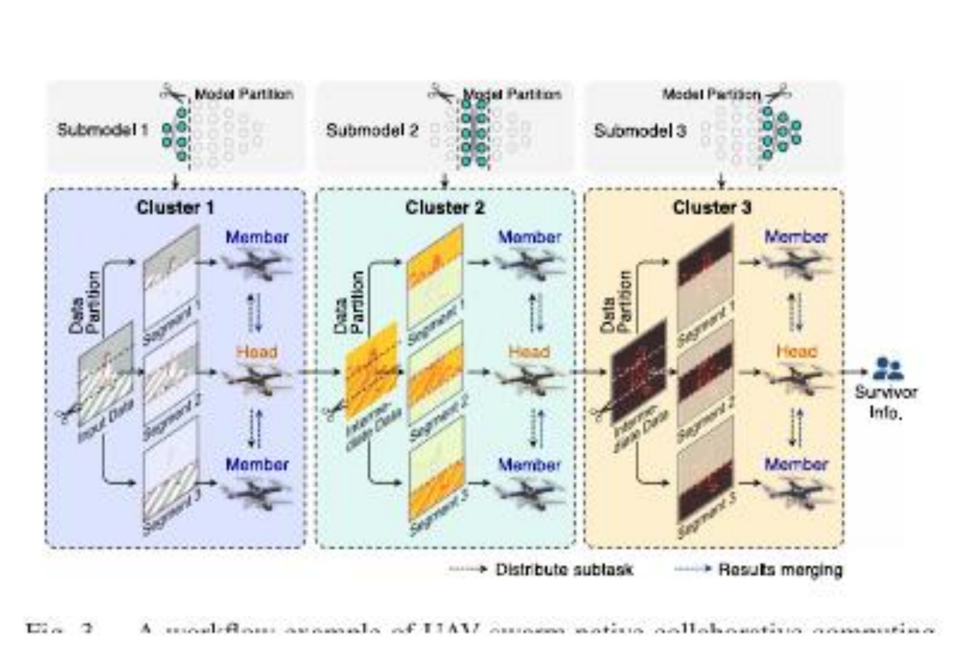


图3：模型在集群之间流水线执行，数据在集群内部并行处理。

Elastic Efficient Scheduler：两层负载均衡

调度器包括三个部分：

- ECLB：External Cluster Load Balancer，按各集群算力分配模型块，让流水线阶段尽量均衡。
- ICLB：Internal Cluster Load Balancer，按 UAV 算力与通信代价划分输入范围，让集群内成员同步完成。
- ES：Elastic Scheduling，检测 cluster head 或成员节点状态变化，并触发重新分配。

调度目标：减少“慢节点拖尾”和流水线空泡

Algorithm 1: Mapping Range Transforming (MRT).

Input: model architecture A, output range R_{out}
Output: input range R_{in}

$\mathbb{R}_{out} \leftarrow \{R^l_{out} \mid R^l_{out} = \varnothing, l \in A\}$
// initialize output range set as empty
$R^{l_O}_{out} \leftarrow R_{out}$
for $l \leftarrow l_O$ **to** l_I **do** // traverse each layer
  $[w_{in}, h_{in}] \leftarrow r\_in(R^l_{out})$ // deduce input range
  if $l = l_I$ **then** // reach input layer
    $R_{in} \leftarrow [w_{in}, h_{in}]$ // calculated input range
    return R_{in}
  foreach *adjacent forward layer* l_a **do**
    if $R^{l_a}_{out} = \varnothing$ **then** // update adjacent forward output
      $R^{l_a}_{out} \leftarrow [w_{in}, h_{in}]$ // range with deduced input
    else // layer output has been calculated
      $R^{l_a}_{out} \leftarrow R^{l_a}_{out} \cup [w_{in}, h_{in}]$ // output range union

Algorithm 2: Internal Cluster Load Balancing (ICLB).

Input: model architecture A, complete input range R_{in}, communication rate r_m between cluster head and each cluster member m, number of cluster members M
Output: input range set $\mathbb{R}_{in}$ for cluster head and cluster members

$r_0 \leftarrow 0$ // set computing rate of itself to 0
for $m \leftarrow 0$ **to** M **do** // traverse each node in the cluster
  $\tau_m \leftarrow \text{Predictor}(A, R_{in})$ // predicted latency
$t_{align} \leftarrow \frac{1}{\sum_{m=0}^{M} \frac{1}{\tau_m}}$ // alignment latency
for $m \leftarrow 0$ **to** M **do** // $m = 0$: cluster head

调度算法细节：从输入范围到弹性恢复

MRT + ICLB：集群内数据并行

- ➢ MRT 从模型输出范围反推最小输入范围，避免卷积/池化边界导致信息缺失。
- ➢ ICLB 先预测每个 UAV 全量输入的时延，再逐步调整各节点输入范围，使完成时间对齐。

ECLB + ES：集群间模型流水线

- ➢ ECLB 根据集群算力控制 submodel 粒度，减少流水线阶段间的等待。
- ➢ **节点失效或恢复时，ES 重新计算可用集群集合并触发 ECLB/ICLB，重调度耗时保持在 1 秒内。**

Algorithm 3: External Cluster Load Balancer (ECLB).

Input: model architecture A, input range R_{in},
number of clusters C,
baseline model inference latency t_c of each cluster c

Output: submodel set $\mathbb{M}$ for each cluster

$\mathbb{M} \leftarrow \{M_c | M_c = \varnothing, \forall c \in [0, C)\}$ *// initialize submodel set*

$\tau_0 \leftarrow$ Predictor(A, R_{in}) *// baseline latency*

for $c \leftarrow 0$ **to** $C - 1$ **do** *// tranverse each cluster*

  $T_c \leftarrow \frac{1}{\sum_{c=0}^{C-1} \frac{1}{t_c}} \cdot \frac{\tau_0}{t_c}$ *// allocated latency for clusters*

  repeat

    add layers to M_c *// contains a few model layers*

    $R_{in}^c \leftarrow$ MRT(A, R_{in}, M_c) *// deduced input range*

    $\tau_c \leftarrow$ Predictor(M_c, R_{in}^c) *// predicted latency*

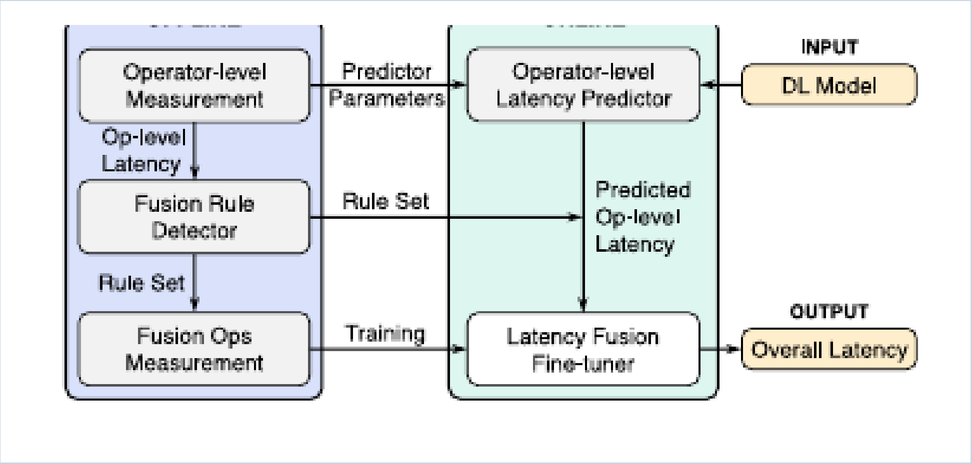
  until *no more reduction on* $|\tau_c - T_c|$ *or model* A *has been partitioned completely*

TABLE IV
RESCHEDULING LATENCY OF ELASTIC EFFICIENT SCHEDULER

Task	Rescheduling latency (ms)
VGG16	989.30
ResNet34	415.58
DarkNet19	616.05

轻量准确的 DL 推理性能预测器

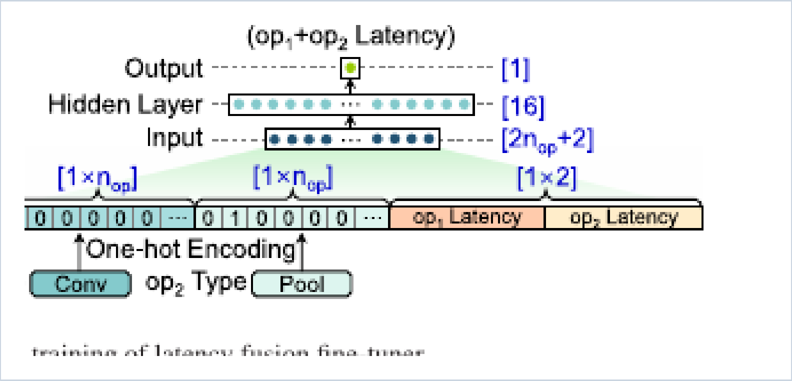
为什什么需要预测器?



- ➢ 调度器必须在线评估“某段模型 + 某份输入”在不同 UAV 上的计算代价。
- ➢ 直接使用复杂 ML 模型做预测太慢，简单 FLOPs 相加又忽略了算子融合和硬件特性。

ASAP 的两阶段策略

- ➢ 离线：测量算子级时延，学习 operator-level predictor 和 fusion rule。
- ➢ **在线：先预测单算子时延，再用 16 神经元的 latency fusion fine-tuner 修正整体时延。**



自适应 Inter-UAV 数据压缩器

压缩对象：中间特征，而不是模型参数

- 集群之间传输的是 submodel 输出的 intermediate data，尺寸大且受链路速率影响。
- ASAP 使用 8-bit 量化 + gzip/LZ4/Bzip2 等无损压缩，降低传输负担。
- **量化 scale 不是固定的，而是根据“数据大小 / 通信速率”自适应选择。**

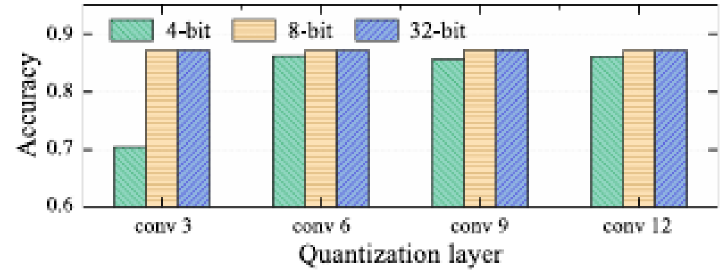


Fig. 8. Inference accuracy under different quantization levels of VGG

关键观察：

- 8-bit 量化接近 32-bit 精度，同时比 32-bit 减少约 75% 数据量。
- **scale 从 0.1 到 0.6 时，中间数据可减少 87.2% - 92.7%，精度下降仍低于 0.15%。**

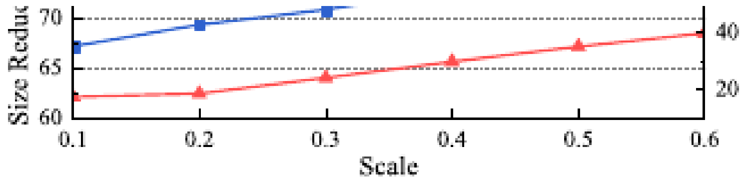


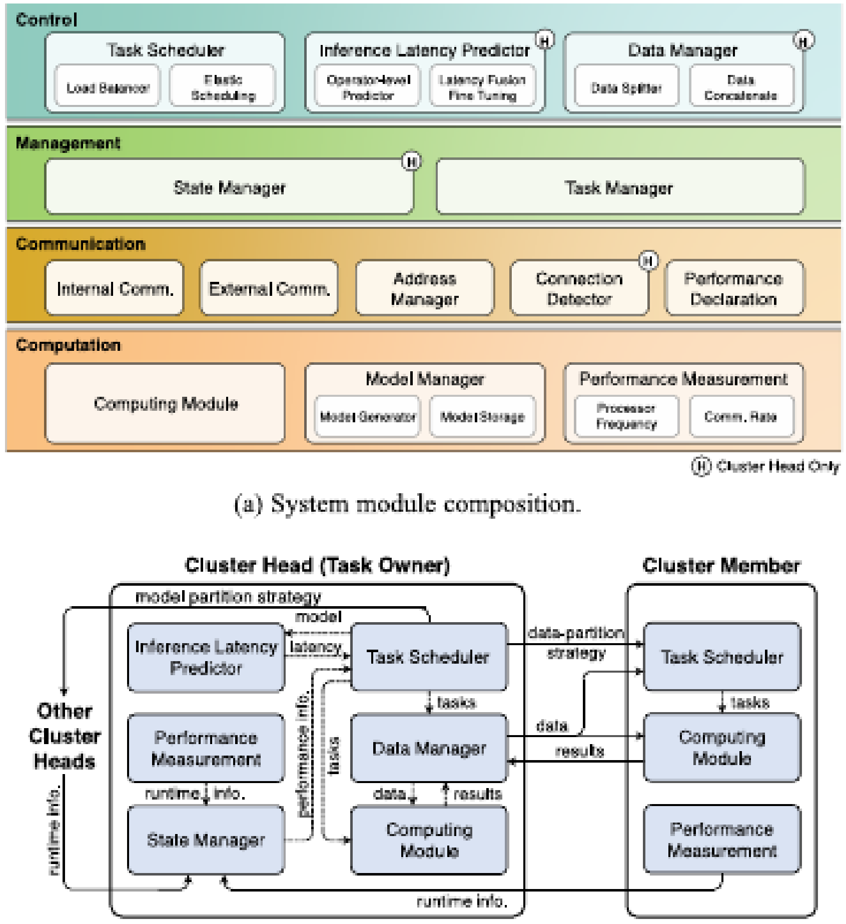
Fig. 9. Data size reduction and error percentage of a random data tenso
different quantization scales.

系统实现：4000+ 行 Python 代码

模块划分

- Control：Task Scheduler、Inference Latency Predictor、Data Manager。
- Management：State Manager 和 Task Manager，维护运行状态与任务生命周期。
- Communication：Internal/External Comm、Address Manager、Connection Detector、Performance Declaration。
- Computation：Computing Module、Model Manager、Performance Measurement。

Cluster head 负责调度和聚合，成员 UAV 负责本地计算与性能上报

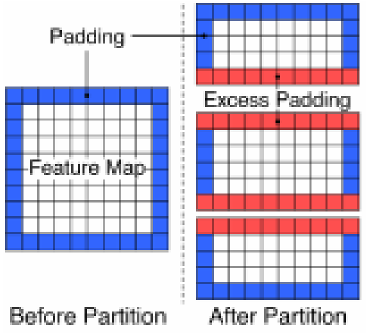


(a) System module composition.

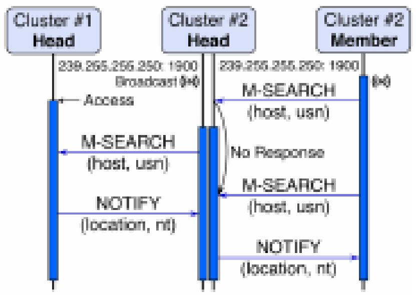
(b) System module relationship.

ig. 10. System module design.

工程细节与实验平台



(a) Padding in data partition.



(b) UAV access protocol.

(a) 24 UAV airborne computers through Wi-Fi connection.

实现技巧:

- ➢ Padding 处理：数据分区可能引入无用边界信息，ASAP 根据 MRT 计算范围自动调整 padding。
- ➢ UAV 接入协议：基于 SSDP 的 M-SEARCH / NOTIFY 消息实现节点发现与集群信息交换。

实验平台：24 台 UAV airborne computers + 5 架真实 UAV；通过 Wi-Fi / Ad-hoc 网络通信。

实验设计：基线、模型与场景

对比系统

- Raw data offloading：将原始数据发回地面站计算。
- Neurosurgeon：边缘设备协同计算代表方法。
- DeepSlicing / MoDNN：地面 IoT 协同计算代表系统。

实验模型

- 中等模型：VGG16、ResNet34、DarkNet19。
- 大型模型：ResNet50、ResNet101、ResNet152，用于测试资源受限下的协同收益。

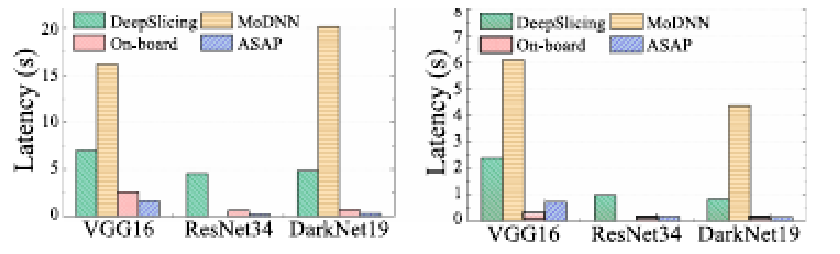
场景	设备构成	目的
11 台机载计算机	7 Nano + 2 NX + 2 TX2	**评估异构节点协同**
5 架真实 UAV	4 Nano + 1 TX2	**验证真实飞行网络**
不同节点数	4 - 20 nodes	**观察扩展性与饱和点**

指标：平均推理时延、单节点计算/内存开销、重调度时延、预测精度、压缩收益

实验结果：ASAP 显著降低推理时延

TABLE III
INFERENCE LATENCY OF ASAP COMPARED WITH RAW DATA OFFLOADING AND EDGE-DEVICE COLLABORATIVE COMPUTING (NEUROSURGEON)

Task	On-board	Raw Data Offloading			Neurosurgeon			ASAP (Ours)				
		20m	50m	100m	20m	50m	100m	4 nodes	8 nodes	12 nodes	16 nodes	20 nodes
ResNet152	—*	5.75	5.79	5.91	7.19	13.21	14.36	2.32	2.07	1.82	1.65	1.32
ResNet101	—	5.28	5.63	5.87	7.07	12.12	12.67	2.13	1.39	1.17	0.93	1.13
ResNet50	1.72	5.26	5.41	5.69	6.09	7.87	8.63	1.28	0.96	0.75	0.94	1.03



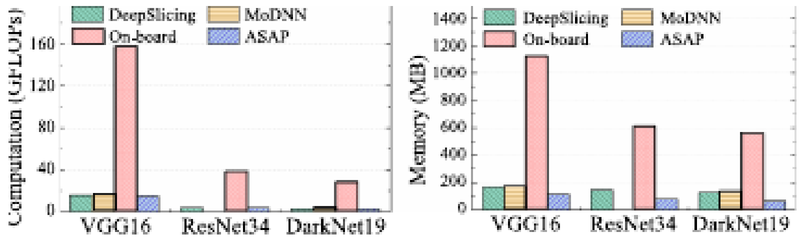
(a) 11 UAV airborne computers. (b) 5 real-world UAVs.

ig. 13. Latency performance of ASAP compared with DeepSlicing,

主要结论

- ➢ 相比 raw data offloading，计算时延最高降低 92.66%。
- ➢ 相比 MoDNN 等地面协同系统，最高降低 98.50%。
- ➢ 随着节点数增加，大模型推理时延下降；但节点过多后收益趋于饱和。

实验结果：开销降低与弹性调度

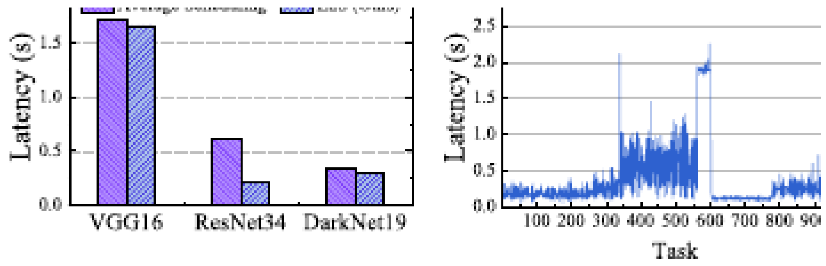


(a) Computing overhead. (b) Memory overhead.

g. 14. Average system overhead per node of ASAP compared with Deep-

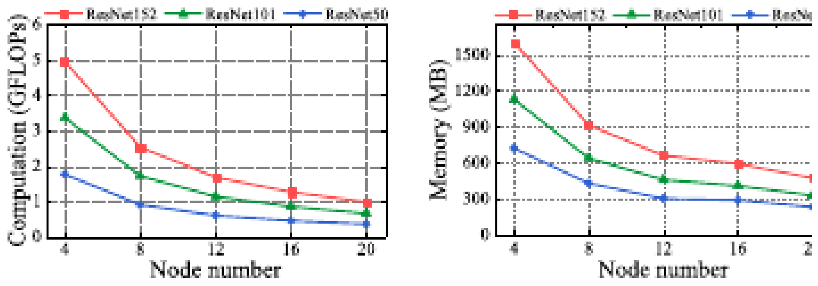
单节点负担：

- **计算开销最高降低 90.56%。**
- **内存开销最高降低 90.02%。**



(a) E2S vs. average scheduling. (b) Unstable node connection

Fig. 16. Performance of E2S scheduling.



(a) Computing overhead. (b) Memory overhead.

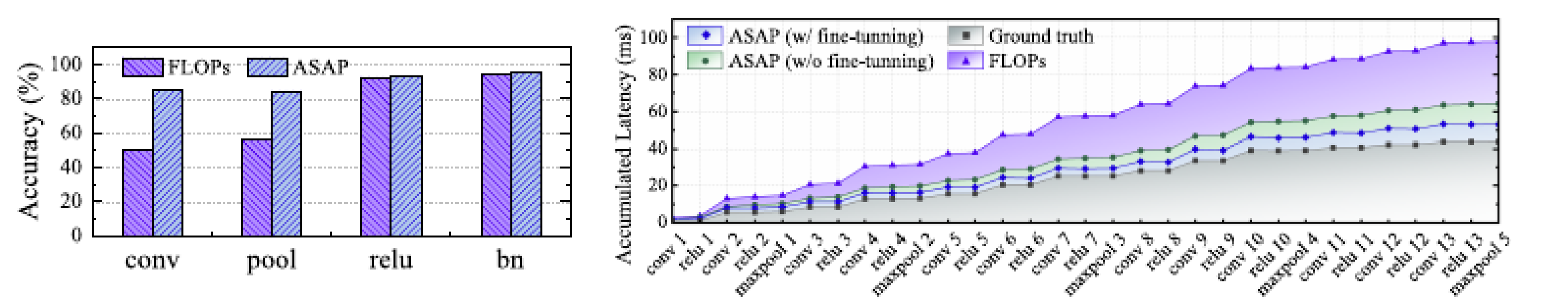
弹性调度：

- E2S 比平均调度更低时延。
- 节点失效后可持续处理新任务。
- **重调度保持在 1 秒内。**

TABLE IV
RESCHEDULING LATENCY OF ELASTIC EFFICIENT SCHEDULER

Task	Rescheduling latency (ms)
VGG16	989.30
ResNet34	415.58
DarkNet19	616.05

实验结果：预测器与压缩器有效性



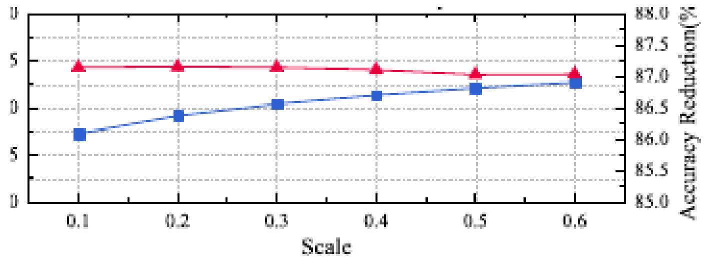
(a) Operator-level inference latency prediction.

(b) Block-level inference latency prediction.

- 算子级预测精度显著高于 FLOPs 方法；block-level 预测中，fusion fine-tuner 进一步降低误差。

TABLE V
PREDICTION LATENCY OF LIGHTWEIGHT ACCURATE PREDICTOR VERSUS NN-METER

Predictor	Prediction Latency (ms)		
	VGG16	ResNet34	DarkNet19
ASAP	12.3	28.0	13.7
nn-Meter	292326.3	15530.9	9057.6



Data size reduction and accuracy reduction under different quantiza-

- **预测速度保持毫秒级，相比 nn-Meter 减少 82.0% - 99.6% 预测时延。**
- **压缩器在保持精度下降低于 0.15% 的同时，将中间数据减少 87.2% - 92.7%。**

总结

- ASAP 面向 UAV swarm 的真实约束，提出“跨集群模型流水线 + 集群内数据并行”的协同计算架构。
- Elastic Scheduler、Latency Predictor、Adaptive Compressor 分别解决调度、预测和通信三类瓶颈。
- 实验表明 ASAP 可显著降低时延与单节点开销，并能在 UAV 节点不可用时保持服务连续性。

汇报结束，感谢！